

Ex. Nos. 7	Performing Code Injection on Application Process Using Ptrace in Kali Linux
Date :	

AIM :

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

SOFTWARE REQUIREMENTS :

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- Programming Language: C
- Debugger / System Tool: Ptrace
- Compiler: GCC
- System Requirements:
 - o RAM: Minimum 4 GB (8 GB recommended)
 - o Disk Space: Minimum 30 GB
 - o Processor: Intel/AMD with Virtualization support

INTRODUCTION :

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers. Understanding ptrace-based code injection helps analyze malware behavior, reverse engineering techniques, and system-level security vulnerabilities. Kali Linux provides a secure environment to study such low-level attacks ethically.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.
5. Compile the program:
6. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
2. ./target
3. Note the Process ID (PID) using:
4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
2. nano injector.c
3. Write ptrace-based code to attach to the target process and modify its memory or registers.
4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

Step 6: Attach to Target Process

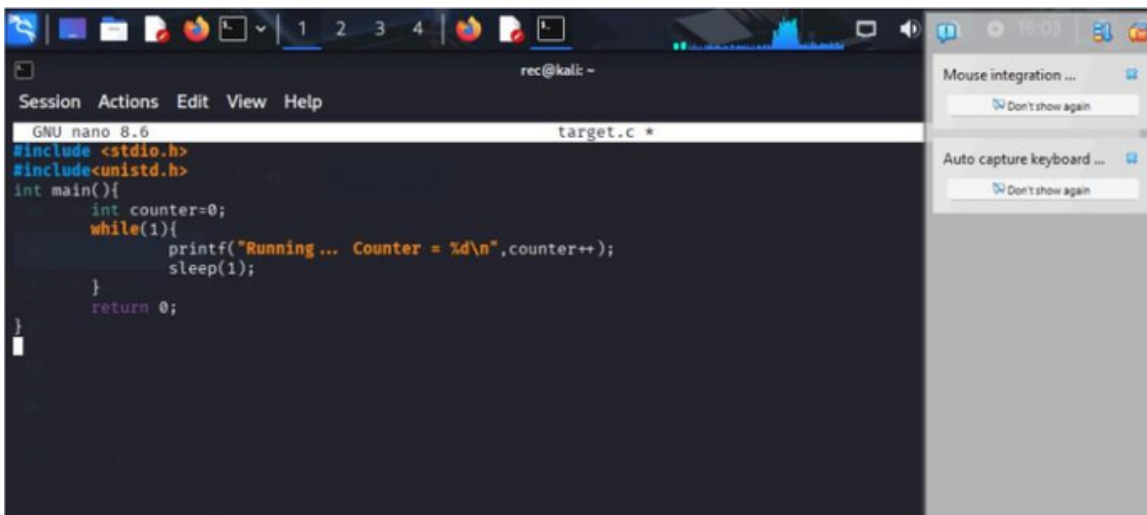
1. Attach injector to target process using PID:
2. sudo ./injector <PID>
3. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.
3. Observe injected behavior.

Output

- Injector successfully attaches to the running process.
- Target application behavior is modified during execution.
- Code injection is observed in real time.



```
GNU nano 8.6 target.c *
#include <stdio.h>
#include <unistd.h>
int main(){
    int counter=0;
    while(1){
        printf("Running... Counter = %d\n",counter++);
        sleep(1);
    }
    return 0;
}
```

```
rec@kali: ~  
Session Actions Edit View Help  
rec@kali: ~  
$ nano target.c  
$ gcc target.c -o target  
$ ./target  
Running ... Counter = 0  
Running ... Counter = 1  
Running ... Counter = 2  
Running ... Counter = 3  
Running ... Counter = 4  
Running ... Counter = 5  
Running ... Counter = 6  
Running ... Counter = 7  
Running ... Counter = 8  
Running ... Counter = 9  
Running ... Counter = 10  
Running ... Counter = 11  
Running ... Counter = 12  
Running ... Counter = 13  
Running ... Counter = 14  
Running ... Counter = 15  
Running ... Counter = 16  
Running ... Counter = 17  
Running ... Counter = 18  
Running ... Counter = 19
```

```
GNU nano 8.6 tracer.c  
#include <sys/ptrace.h>  
#include <sys/types.h>  
#include <sys/wait.h>  
#include <sys/user.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int main(int argc, char *argv[]) {  
    if(argc < 2) {  
        printf("Usage: %s <PID>\n", argv[0]);  
        return 1;  
    }  
  
    pid_t pid = atoi(argv[1]);  
  
    // Attach to process  
    ptrace(PTRACE_ATTACH, pid, NULL, NULL);  
    wait(NULL);  
  
    printf("Attached to process %d\n", pid);  
  
    struct user_regs_struct regs;  
  
    // Get registers  
    ptrace(PTRACE_GETREGS, pid, NULL, &regs);  
  
    printf("Instruction Pointer (RIP): %llx\n", regs.rip);  
  
    // Detach  
    ptrace(PTRACE_DETACH, pid, NULL, NULL);  
  
    printf("Detached successfully\n");  
  
    return 0;  
}
```

```
rec@kali: ~  
$ ps aux | grep target  
rec 13144 0.0 0.0 2568 1652 pts/0 S+ 16:05 0:00 ./target  
rec 13379 0.0 0.1 6544 2220 pts/1 S+ 16:06 0:00 grep --color=auto target  
  
$ nano tracer.c  
  
$ gcc tracer.c -o tracer  
  
$ sudo ./tracer 13144  
[sudo] password for rec:  
Attached to process 13144  
Instruction Pointer (RIP): 7f73e4b0b687  
Detached successfully  
  
$
```

RESULT :

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

Reference

<https://man7.org/linux/man-pages/man2/ptrace.2.html>

⚠ Note

This experiment is conducted strictly for academic and ethical purposes within a controlled laboratory environment.